

AI READY PREMIUM KIT

Soluzioni Commentate

Codice, spiegazione, output atteso, errore comune e variante per ogni esercizio

150 soluzioni complete

Materiale bonus di "Python da Zero con Google Colab" - Tania Cilio

Come usare le soluzioni

Per ogni esercizio del Workbook qui trovi: la soluzione in codice, la spiegazione del perché funziona, l'output atteso, l'errore comune da evitare e una variante da provare per spingerti oltre.

Guarda la soluzione solo dopo aver provato. Non esiste un solo modo giusto: se il tuo codice dà il risultato corretto, va bene anche se diverso dal mio. Qui trovi una versione semplice e leggibile, pensata per imparare.

Soluzioni - Base

Esercizio 1. Primo saluto

Soluzione:

```
print("Ciao, sto imparando Python!")
```

Output atteso:

```
Ciao, sto imparando Python!
```

Perché funziona: print() mostra a schermo ciò che metti tra parentesi. Il testo va tra virgolette.

Errore comune: Dimenticare le virgolette: print(Ciao) dà errore perché Python cerca una variabile di nome Ciao.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 2. Tre righe

Soluzione:

```
print("Anna")  
print("Roma")  
print("Leggere")
```

Output atteso:

```
Anna  
Roma  
Leggere
```

Perché funziona: Ogni print() va a capo da solo: tre print() producono tre righe.

Errore comune: Mettere tutto in un solo print con le virgole stampa tutto su una riga sola.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 3. Riga vuota

Soluzione:

```
print("INIZIO")  
print()  
print("FINE")
```

Output atteso:

```
INIZIO  
  
FINE
```

Perché funziona: print() vuoto serve proprio a creare spazio verticale tra due testi.

Errore comune: Scrivere print(" ") con uno spazio non è la stessa cosa di print() vuoto, anche se sembra simile.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 4. Etichetta e valore

Soluzione:

```
print("Il mio numero fortunato è:", 7)
```

Output atteso:

```
Il mio numero fortunato è: 7
```

Perché funziona: La virgola dentro print() unisce più elementi separandoli con uno spazio.

Errore comune: Scrivere il 7 tra virgolette lo trasforma in testo: funziona uguale ma non è più un numero.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 5. Commenti

Soluzione:

```
# Questo programma stampa un saluto di pronto  
print("Pronto!")
```

Output atteso:

```
Pronto!
```

Perché funziona: Tutto ciò che segue # sulla stessa riga viene ignorato da Python: serve a te, non al computer.

Errore comune: Pensare che il commento venga stampato: no, i commenti non producono output.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 6. Le mie variabili

Soluzione:

```
nome = "Anna"
eta = 30
print(nome)
print(eta)
```

Output atteso:

```
Anna
30
```

Perché funziona: Una variabile è un contenitore con un nome: a sinistra il nome, a destra il valore.

Errore comune: Mettere l'età tra virgolette la rende testo invece che numero.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 7. Presentazione

Soluzione:

```
nome = "Anna"
eta = 30
print(f"Mi chiamo {nome} e ho {eta} anni.")
```

Output atteso:

```
Mi chiamo Anna e ho 30 anni.
```

Perché funziona: La f-string (la f prima delle virgolette) permette di inserire variabili dentro il testo con le graffe.

Errore comune: Dimenticare la f davanti alle virgolette: le graffe verrebbero stampate così come sono.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 8. Aggiornare un valore

Soluzione:

```
punteggio = 0
print(punteggio)
punteggio = punteggio + 10
print(punteggio)
```

Output atteso:

```
0
10
```

Perché funziona: Una variabile può cambiare valore: `punteggio = punteggio + 10` prende il vecchio valore e ne mette uno nuovo.

Errore comune: Scrivere solo `punteggio + 10` senza riassegnare: il calcolo avviene ma il risultato si perde.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 9. I quattro tipi

Soluzione:

```
testo = "ciao"
intero = 42
decimale = 3.14
vero_falso = True
print(type(testo))
print(type(intero))
print(type(decimale))
print(type(vero_falso))
```

Output atteso:

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

Perché funziona: I quattro tipi base: str (testo), int (interi), float (decimali), bool (vero/falso).

Errore comune: Scrivere true minuscolo: in Python si scrive True con la T maiuscola.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 10. Decimali col punto

Soluzione:

```
prezzo = 19.99
```

```
print(prezzo)
```

Output atteso:

```
19.99
```

Perché funziona: In Python i decimali si scrivono con il punto: 19.99, non 19,99.

Errore comune: Scrivere 19,99 con la virgola: Python lo interpreta come due numeri (una tupla), non un decimale.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 11. Scambio di valori

Soluzione:

```
a = 10
b = 20
a, b = b, a
print(f"a = {a}, b = {b}")
```

Output atteso:

```
a = 20, b = 10
```

Perché funziona: Python permette di scambiare due variabili in una riga sola: a, b = b, a.

Errore comune: Fare a = b e poi b = a senza variabile temporanea: si perde il valore originale di a.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 12. Nomi validi

Soluzione:

```
nome_utente = "Anna"
prezzo_unitario = 19.99
anno_nascita = 1994
print(nome_utente, prezzo_unitario, anno_nascita)
```

Output atteso:

```
(dipende dai dati inseriti)
```

Perché funziona: I nomi di variabile non possono avere spazi: si usa l'underscore (snake_case) per separare le parole.

Errore comune: Usare lo spazio (nome utente) o iniziare con un numero (1nome) dà errore di sintassi.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 13. Maiuscole e minuscole

Soluzione:

```
frase = "python e fantastico"
print(frase.upper())
print(frase.lower())
```

Output atteso:

```
PYTHON E FANTASTICO
python e fantastico
```

Perché funziona: .upper() restituisce il testo in maiuscolo, .lower() in minuscolo. Non cambiano la variabile originale.

Errore comune: Pensare che frase.upper() cambi la variabile frase: no, restituisce un nuovo testo. Per salvarlo devi riassegnarlo.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 14. Togliere gli spazi

Soluzione:

```
testo = " ciao "
print(testo.strip())
```

Output atteso:

```
ciao
```

Perché funziona: .strip() rimuove gli spazi (e gli a-capo) all'inizio e alla fine, ma non quelli in mezzo.

Errore comune: Pensare che strip() tolga tutti gli spazi: toglie solo quelli ai bordi, non quelli interni.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 15. Unire parole

Soluzione:

```
nome = "Anna"
cognome = "Rossi"
print(nome + " " + cognome)
```

Output atteso:

```
Anna Rossi
```

Perché funziona: Il + tra stringhe le attacca. Per avere lo spazio devi aggiungerlo tu: nome + " " + cognome.

Errore comune: Scrivere nome + cognome senza spazio produce AnnaRossi tutto attaccato.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 16. Contare i caratteri

Soluzione:

```
parola = "programmazione"
print(len(parola))
```

Output atteso:

```
14
```

Perché funziona: len() restituisce il numero di caratteri di una stringa (o elementi di una lista).

Errore comune: Contare a mano: inutile e si sbaglia. len() lo fa per te.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 17. Sostituire

Soluzione:

```
frase = "Mi piace il caffè"
print(frase.replace("caffè", "tè"))
```

Output atteso:

```
Mi piace il tè
```

Perché funziona: .replace(vecchio, nuovo) sostituisce tutte le occorrenze di una parola con un'altra.

Errore comune: L'ordine conta: .replace(nuovo, vecchio) farebbe il contrario di quello che vuoi.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 18. f-string con calcolo

Soluzione:

```
prezzo = 19.99
print(f"Il prezzo con IVA al 22% è: {prezzo * 1.22:.2f}")
```

Output atteso:

```
Il prezzo con IVA al 22% è: 24.39
```

Perché funziona: Dentro le graffe di una f-string puoi mettere calcoli. :.2f arrotonda a 2 decimali.

Errore comune: Dimenticare :.2f mostra tantissimi decimali (24.3878...) poco leggibili.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 19. Saluto personalizzato

Soluzione:

```
nome = input("Come ti chiami? ")
print(f"Ciao, {nome}!")
```

Output atteso:

```
(se scrive Anna) Ciao, Anna!
```

Perché funziona: input() mostra un messaggio e aspetta che l'utente scriva qualcosa, restituendo sempre del testo.

Errore comune: Dimenticare che input() restituisce sempre testo: anche se scrivi 30, è la stringa "30", non il numero.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 20. Età tra un anno

Soluzione:

```
eta = int(input("Quanti anni hai? "))
print(f"L'anno prossimo avrai {eta + 1} anni")
```

Output atteso:

```
(se scrive 30) L'anno prossimo avrai 31 anni
```

Perché funziona: int() converte il testo dell'input in numero intero, così puoi fare i calcoli.

Errore comune: Fare eta + 1 senza int(): dà errore perché non puoi sommare 1 a del testo.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 21. Prezzo con IVA da input

Soluzione:

```
prezzo = float(input("Inserisci il prezzo: "))
print(f"Prezzo con IVA: {prezzo * 1.22:.2f}")
```

Output atteso:

```
(se scrive 100) Prezzo con IVA: 122.00
```

Perché funziona: float() converte il testo in numero decimale: serve quando l'utente può inserire prezzi con la virgola.

Errore comune: Usare int() su un prezzo come 19.99 dà errore: int vuole numeri interi.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 22. Somma di due numeri

Soluzione:

```
a = int(input("Primo numero: "))
b = int(input("Secondo numero: "))
print(f"La somma è: {a + b}")
```

Output atteso:

```
(se scrive 4 e 5) La somma è: 9
```

Perché funziona: Servono due input separati, ciascuno convertito con int().

Errore comune: Dimenticare int() su uno dei due: 4 + 5 come testo diventa 45 (li attacca) invece di 9.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 23. Nome e cognome

Soluzione:

```
nome = input("Nome: ")
cognome = input("Cognome: ")
print(f"{nome} {cognome}".upper())
```

Output atteso:

```
(se scrive Anna Rossi) ANNA ROSSI
```

Perché funziona: Puoi applicare .upper() al risultato di una f-string mettendola tra parentesi.

Errore comune: Applicare .upper() solo a nome dimentica il cognome.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 24. Calcolatrice base

Soluzione:

```
a = 17
b = 5
print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

Output atteso:

```
22
12
85
3.4
```

Perché funziona: Gli operatori base: + - * /. La divisione / restituisce sempre un decimale.

Errore comune: Aspettarsi che 17 / 5 dia un intero: dà 3.4, perché / produce sempre un float.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 25. Divisione intera e resto

Soluzione:

```
a = 17
b = 5
print(a // b)
print(a % b)
```

Output atteso:

```
3
2
```

Perché funziona: // è la divisione intera (floor division): con numeri positivi dà il quoziente intero. % dà il resto.

Errore comune: Confondere / e //: la prima dà 3.4, la seconda dà 3.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 26. Pari o dispari

Soluzione:

```
n = int(input("Numero: "))
if n % 2 == 0:
    print(f"{n} è pari")
else:
    print(f"{n} è dispari")
```

Output atteso:

```
(se scrive 8) 8 è pari
```

Perché funziona: $n \% 2$ dà 0 per i numeri pari, 1 per i dispari: è il trucco classico.

Errore comune: Usare = invece di == nel confronto: = assegna, == confronta.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 27. Potenza e radice

Soluzione:

```
print(2 ** 10)
print(81 ** 0.5)
```

Output atteso:

```
1024
9.0
```

Perché funziona: ** eleva a potenza. Elevare a 0.5 equivale a fare la radice quadrata.

Errore comune: Cercare una funzione complicata per la radice: $81^{0.5}$ basta e avanza.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 28. Media di tre numeri

Soluzione:

```
v1 = 28
v2 = 30
v3 = 26
media = (v1 + v2 + v3) / 3
print(f"{media:.2f}")
```

Output atteso:

```
28.00
```

Perché funziona: Le parentesi servono: senza, Python dividerebbe solo $v3$ per 3 (priorità degli operatori).

Errore comune: Scrivere $v1 + v2 + v3 / 3$ senza parentesi: divide solo $v3$, dando un risultato sbagliato.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 29. Operatori abbreviati

Soluzione:

```
totale = 100
totale += 50
print(totale)
totale /= 2
print(totale)
```

Output atteso:

```
150
75.0
```

Perché funziona: $\text{totale} += 50$ è la scorciatoia di $\text{totale} = \text{totale} + 50$. Comodo e leggibile.

Errore comune: Pensare che += crei una nuova variabile: modifica quella esistente.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 30. Maggiorennne

Soluzione:

```
eta = int(input("Età: "))
if eta >= 18:
    print("Maggiorennne")
```



```
else:
    print("Minorenne")
```

Output atteso:

```
(se 20) Maggiorennne
```

Perché funziona: if/else: se la condizione è vera esegue il primo blocco, altrimenti il secondo. L'indentazione conta.

Errore comune: Dimenticare i due punti dopo if o non indentare il blocco: errore di sintassi.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 31. Voto in lettere

Soluzione:

```
voto = int(input("Voto: "))
if voto >= 90:
    print("A")
elif voto >= 80:
    print("B")
elif voto >= 70:
    print("C")
else:
    print("Bocciato")
```

Output atteso:

```
(se 85) B
```

Perché funziona: elif permette di controllare più condizioni in sequenza: Python si ferma alla prima vera.

Errore comune: Mettere le condizioni in ordine sbagliato (prima >=70) classificherebbe tutto come C.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 32. Numero positivo, negativo o zero

Soluzione:

```
n = int(input("Numero: "))
if n > 0:
    print("Positivo")
elif n < 0:
    print("Negativo")
else:
    print("Zero")
```

Output atteso:

```
(se -5) Negativo
```

Perché funziona: Tre casi, tre rami: if, elif, else. L'else cattura il caso rimanente (zero).

Errore comune: Usare due if separati invece di elif funziona ma è meno chiaro e più lento.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 33. Sconto condizionale

Soluzione:

```
imp = float(input("Importo: "))
if imp > 100:
    imp = imp * 0.9
print(f"Totale: {imp:.2f}")
```

Output atteso:

```
(se 200) Totale: 180.00
```

Perché funziona: Puoi modificare una variabile dentro un if: lo sconto si applica solo se la condizione è vera.

Errore comune: Mettere il print dentro l'if: così non stamperebbe nulla quando l'importo è sotto 100.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 34. Accesso con due condizioni

Soluzione:

```
eta = int(input("Età: "))
patente = input("Hai la patente? (si/no) ")
if eta >= 18 and patente == "si":
    print("Può guidare")
else:
    print("Non può guidare")
```

Output atteso:

(se 20 e sì) Può guidare

Perché funziona: and richiede che entrambe le condizioni siano vere. Basta che una sia falsa e il risultato è falso.

Errore comune: Confrontare patente == sì senza virgolette: si verrebbe letto come nome di variabile.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 35. Anno bisestile (semplificato)

Soluzione:

```
anno = int(input("Anno: "))
if anno % 4 == 0:
    print(f"{anno} è divisibile per 4")
else:
    print(f"{anno} non è divisibile per 4")
```

Output atteso:

(se 2024) 2024 è divisibile per 4

Perché funziona: Il test di divisibilità si fa sempre con il modulo %: resto 0 significa divisibile.

Errore comune: Usare / invece di % per testare la divisibilità: sbagliato, / non dice se è divisibile.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 36. Biglietto da visita

Soluzione:

```
nome = "Anna Rossi"
professione = "Formatrice"
citta = "Milano"
email = "anna@example.com"
print(f"Nome: {nome}")
print(f"Professione: {professione}")
print(f"Città: {citta}")
print(f"Email: {email}")
```

Output atteso:

Nome: Anna Rossi
Professione: Formatrice
Città: Milano
Email: anna@example.com

Perché funziona: Ogni f-string unisce un'etichetta fissa al valore della variabile. Le 4 print() producono 4 righe.

Errore comune: Mettere le variabili tra virgolette dentro la f-string: stamperebbe il nome della variabile, non il valore.

Variante da provare: Aggiungi una quinta riga con il sito web, e prova a stampare nome e città sulla stessa riga.

Esercizio 37. Conto delle ore

Soluzione:

```
ore_giorno = 7
giorni = 5
paga_oraria = 12
ore_totali = ore_giorno * giorni
compenso = ore_totali * paga_oraria
print(f"Ore totali: {ore_totali}")
print(f"Compenso: {compenso} euro")
```

Output atteso:

Ore totali: 35
Compenso: 420 euro

Perché funziona: Scomponi il calcolo in due passi: prima le ore totali, poi il compenso. È più leggibile che fare tutto in una riga.

Errore comune: Fare ore_giorno * giorni * paga in una riga senza variabili intermedie: funziona ma è meno chiaro e più facile da sbagliare.

Variante da provare: Aggiungi una percentuale di tasse del 20% e stampa il compenso netto.

Esercizio 38. Temperatura da Celsius a Fahrenheit

Soluzione:

```
celsius = 25
fahrenheit = celsius * 9/5 + 32
print(f"{celsius} C = {fahrenheit} F")
```

Output atteso:

```
25 C = 77.0 F
```

Perché funziona: La formula si traduce direttamente in Python. Grazie alla priorità degli operatori, * e / vengono prima di +, quindi non servono parentesi qui.

Errore comune: Scrivere `celsius * 9 / (5 + 32)`: le parentesi sbagliate cambierebbero il risultato.

Variante da provare: Fai anche la conversione inversa: da Fahrenheit a Celsius con `C = (F - 32) * 5/9`.

Esercizio 39. Secondi in ore, minuti, secondi

Soluzione:

```
totale = 3725
ore = totale // 3600
resto = totale % 3600
minuti = resto // 60
secondi = resto % 60
print(f"{ore}h {minuti}m {secondi}s")
```

Output atteso:

```
1h 2m 5s
```

Perché funziona: // dà il quoziente intero, % il resto. Prima estrai le ore, poi dal resto i minuti, poi i secondi.

Errore comune: Usare / invece di //: otterresti numeri decimali invece di ore intere.

Variante da provare: Aggiungi i giorni: 86400 secondi fanno un giorno.

Esercizio 40. Iniziali di un nome

Soluzione:

```
nome = "Anna"
cognome = "Rossi"
iniziali = nome[0] + "." + cognome[0] + "."
print(iniziali)
```

Output atteso:

```
A.R.
```

Perché funziona: `nome[0]` prende la prima lettera. Le concateni con i punti usando il +.

Errore comune: Usare `nome[1]` pensando che il primo carattere sia all'indice 1: in Python si parte da 0.

Variante da provare: Rendi le iniziali sempre maiuscole con `.upper()`, anche se il nome è scritto minuscolo.

Esercizio 41. Contare le vocali

Soluzione:

```
parola = "programmazione"
vocali = "aeiou"
contatore = 0
for lettera in parola:
    if lettera in vocali:
        contatore += 1
print(f"Vocali: {contatore}")
```

Output atteso:

```
Vocali: 6
```

Perché funziona: Scorri la parola lettera per lettera. 'lettera in vocali' è vero se quella lettera è una vocale. L'accumulatore conta.

Errore comune: Dimenticare di inizializzare `contatore = 0` prima del ciclo: darebbe un `NameError`.

Variante da provare: Conta invece le consonanti, oppure chiedi la parola all'utente con `input()`.

Esercizio 42. Parola palindroma

Soluzione:

```
parola = "anna"
rovescio = parola[::-1]
if parola == rovescio:
    print(f"{parola} è palindroma")
else:
    print(f"{parola} non è palindroma")
```

Output atteso:

```
anna è palindroma
```

Perché funziona: `[::-1]` crea la stringa al contrario. Se è uguale all'originale, è palindroma.

Errore comune: Confrontare con `=` invece di `==`: il primo assegna, il secondo confronta.

Variante da provare: Rendi il confronto insensibile alle maiuscole con `.lower()`, così 'Anna' risulta palindroma.

Esercizio 43. Maschera una password

Soluzione:

```
password = "segreto"
mascherata = "*" * (len(password) - 1) + password[-1]
print(mascherata)
```

Output atteso:

```
*****o
```

Perché funziona: `'*'` moltiplicato per un numero ripete l'asterisco. `len(password)-1` dice quanti asterischi servono prima dell'ultima lettera.

Errore comune: Usare `len(password)` invece di `len(password)-1`: avresti un asterisco di troppo.

Variante da provare: Mostra invece i primi due caratteri e maschera il resto.

Esercizio 44. Classifica BMI semplificata

Soluzione:

```
indice = 22.5
if indice < 18.5:
    print("Basso")
elif indice <= 25:
    print("Normale")
else:
    print("Alto")
```

Output atteso:

```
Normale
```

Perché funziona: `elif` controlla le condizioni in ordine. Quando una è vera, le altre vengono saltate.

Errore comune: Mettere le condizioni in ordine sbagliato (prima il caso 'Alto') classificherebbe male i valori.

Variante da provare: Chiedi il valore all'utente con `float(input())` e gestisci anche il caso di valore negativo (non valido).

Esercizio 45. Anno bisestile completo

Soluzione:

```
anno = 2000
if (anno % 4 == 0 and anno % 100 != 0) or (anno % 400 == 0):
    print(f"{anno} è bisestile")
else:
    print(f"{anno} non è bisestile")
```

Output atteso:

```
2000 è bisestile
```

Perché funziona: Le parentesi raggruppano le condizioni. La regola reale dei bisestili richiede sia `and` sia `or` insieme.

Errore comune: Dimenticare le parentesi attorno alle due condizioni: l'ordine di `and/or` cambierebbe il risultato.

Variante da provare: Prova con 1900 (non bisestile) e 2024 (bisestile) per verificare la regola.

Esercizio 46. Prezzo del biglietto per età

Soluzione:

```
eta = 70
if eta < 6:
    prezzo = 0
elif eta <= 18:
    prezzo = 5
elif eta <= 65:
    prezzo = 10
else:
    prezzo = 7
print(f"Prezzo biglietto: {prezzo} euro")
```

Output atteso:

```
Prezzo biglietto: 7 euro
```

Perché funziona: Le fasce si controllano in ordine. Quando trovi quella giusta, assegna il prezzo e salti il resto.

Errore comune: Sovrapporre le fasce o lasciarne dei buchi: controlla che ogni età ricada in una sola fascia.

Variante da provare: Chiedi l'età con `input()` e aggiungi uno sconto del 50% per i gruppi (chiedi se è un gruppo).

Esercizio 47. Resto della spesa

Soluzione:

```
costo = 13.50
pagato = 20
resto = pagato - costo
print(f"Resto: {resto:.2f} euro")
```

Output atteso:

```
Resto: 6.50 euro
```

Perché funziona: Una semplice sottrazione. `:.2f` formatta il resto a due decimali, come si fa coi soldi.

Errore comune: Dimenticare `:.2f`: il resto potrebbe mostrarsi come 6.5 invece di 6.50.

Variante da provare: Gestisci il caso in cui il pagato è inferiore al costo (mancano soldi).

Esercizio 48. Area e perimetro del rettangolo

Soluzione:

```
base = 8
altezza = 5
area = base * altezza
perimetro = 2 * (base + altezza)
print(f"Area: {area}")
print(f"Perimetro: {perimetro}")
```

Output atteso:

```
Area: 40
Perimetro: 26
```

Perché funziona: Due formule distinte sulle stesse variabili. Le parentesi nel perimetro sono necessarie per sommare prima.

Errore comune: Scrivere `2 * base + altezza` senza parentesi: calcolerebbe $(2 * \text{base}) + \text{altezza}$, sbagliato.

Variante da provare: Chiedi base e altezza all'utente e calcola anche la diagonale (radice di $\text{base}^2 + \text{altezza}^2$).

Esercizio 49. Maiuscolo, minuscolo, titolo

Soluzione:

```
frase = "imparare python e divertente"
print(frase.upper())
print(frase.lower())
print(frase.title())
```

Output atteso:

```
IMPARARE PYTHON E DIVERTENTE
imparare python e divertente
Imparare Python E Divertente
```

Perché funziona: `.title()` mette maiuscola l'iniziale di ogni parola: utile per nomi e titoli.

Errore comune: Pensare che questi metodi cambino la variabile: restituiscono un nuovo testo.

Variante da provare: Conta quante parole ci sono nella frase con `len(frase.split())`.

Esercizio 50. Sconto percentuale

Soluzione:

```
prezzo = 80
perc = 25
sconto = prezzo * perc / 100
finale = prezzo - sconto
print(f"Sconto: {sconto:.2f} euro")
print(f"Prezzo finale: {finale:.2f} euro")
```

Output atteso:

```
Sconto: 20.00 euro
Prezzo finale: 60.00 euro
```

Perché funziona: Calcoli lo sconto come frazione del prezzo, poi lo sottrai. Due passaggi chiari.

Errore comune: Scrivere `prezzo * perc` senza dividere per 100: otterresti uno sconto enorme.

Variante da provare: Aggiungi un secondo sconto del 10% sul prezzo già scontato.

Esercizio 51. Calcolo dell'IVA

Soluzione:

```
netto = 50
iva = netto * 22 / 100
totale = netto + iva
print(f"IVA: {iva:.2f} euro")
print(f"Totale: {totale:.2f} euro")
```

Output atteso:

```
IVA: 11.00 euro
Totale: 61.00 euro
```

Perché funziona: Calcoli prima l'IVA come percentuale del netto, poi la sommi. Due passaggi chiari e verificabili.

Errore comune: Calcolare `totale = netto * 22/100` dimenticando di sommare il netto.

Variante da provare: Parti invece dal prezzo ivato e calcola lo scorporo dell'IVA.

Esercizio 52. Doppio, triplo, quadruplo

Soluzione:

```
n = 7
print(f"Doppio: {n*2}")
print(f"Triplo: {n*3}")
print(f"Quadruplo: {n*4}")
```

Output atteso:

```
Doppio: 14
Triplo: 21
Quadruplo: 28
```

Perché funziona: Calcoli direttamente dentro la f-string. Semplice e leggibile.

Errore comune: Scrivere `n*2` fuori dalle graffe nella f-string: comparirebbe il testo `'n*2'`.

Variante da provare: Chiedi il numero all'utente e stampane anche la metà.

Esercizio 53. Saluto in base all'ora

Soluzione:

```
ora = 15
if 5 <= ora <= 12:
    print("Buongiorno")
elif 13 <= ora <= 17:
    print("Buon pomeriggio")
elif 18 <= ora <= 22:
    print("Buonasera")
else:
    print("Buonanotte")
```

Output atteso:

```
Buon pomeriggio
```

Perché funziona: Python permette di scrivere `5 <= ora <= 12` per controllare un intervallo in modo naturale.

Errore comune: Lasciare buchi tra le fasce (es. saltare l'ora 12) così alcune ore non rientrano in nessun saluto.

Variante da provare: Chiedi l'ora con `input()` e gestisci valori non validi (sopra 23 o negativi).

Esercizio 54. Numero positivo, pari e altro

Soluzione:

```
n = -4
if n >= 0:
    print("Positivo o zero")
else:
    print("Negativo")
if n % 2 == 0:
    print("Pari")
else:
    print("Dispari")
```

Output atteso:

```
Negativo
Pari
```

Perché funziona: Due controlli indipendenti, quindi due if separati (non elif): le due informazioni sono entrambe valide.

Errore comune: Usare elif tra i due controlli: sono indipendenti, vanno valutati entrambi.

Variante da provare: Aggiungi un terzo controllo: se il numero è multiplo di 5.

Esercizio 55. Conta alla rovescia con messaggio

Soluzione:

```
for i in range(10, 0, -1):
    print(i)
print("Partenza!")
```

Output atteso:

```
10
9
...
1
Partenza!
```

Perché funziona: Il passo -1 fa contare all'indietro. Lo 0 è escluso, quindi si ferma a 1.

Errore comune: Scrivere range(10, 1, -1): si fermerebbe a 2, saltando l'1.

Variante da provare: Stampa solo i numeri pari del conto alla rovescia.

Esercizio 56. Lunghezza media delle parole

Soluzione:

```
frase = "il gatto nero dorme"
parole = frase.split()
totale_lettere = sum(len(p) for p in parole)
media = totale_lettere / len(parole)
print(f"Lunghezza media: {media:.2f}")
```

Output atteso:

```
Lunghezza media: 3.75
```

Perché funziona: sum(len(p) for p in parole) somma le lunghezze di tutte le parole. Diviso il numero di parole dà la media.

Errore comune: Dividere per len(frase) invece che per len(parole): conteresti i caratteri, non le parole.

Variante da provare: Trova la parola più lunga e la più corta.

Esercizio 57. Verifica email semplice

Soluzione:

```
email = "anna@example.com"
if "@" in email and "." in email:
    print("Sembra valida")
else:
    print("Non valida")
```

Output atteso:

```
Sembra valida
```

Perché funziona: 'in' verifica se un carattere è presente. La validazione vera è più complessa, ma questo controllo base copre i casi ovvi.

Errore comune: Pensare che questo basti per validare davvero un'email: è solo un controllo minimo.

Variante da provare: Aggiungi il controllo che la @ venga prima del punto.

Esercizio 58. Etichetta prezzo allineata

Soluzione:

```
prodotti = [("Pane", 1.2), ("Prosciutto", 18.5), ("Sale", 0.45)]
for nome, prezzo in prodotti:
    print(f"{nome:<12}{prezzo:>8.2f} euro")
```

Output atteso:

```
Pane          1.20 euro
Prosciutto    18.50 euro
Sale          0.45 euro
```

Perché funziona: :<12 allinea il nome a sinistra in 12 spazi, :>8.2f il prezzo a destra: i prezzi risultano incolonnati.

Errore comune: Dimenticare l'allineamento: i prezzi risulterebbero disordinati e poco leggibili.

Variante da provare: Aggiungi una riga col totale dei tre prezzi, allineata come gli altri.

Esercizio 59. Conversione km in miglia

Soluzione:

```
km = 10
miglia = km * 0.621
print(f"{km} km = {miglia:.2f} miglia")
```

Output atteso:

```
10 km = 6.21 miglia
```

Perché funziona: Una moltiplicazione diretta. :.2f arrotonda il risultato a due decimali.

Errore comune: Confondere il fattore di conversione o invertire l'operazione.

Variante da provare: Fai anche la conversione inversa da miglia a km.

Esercizio 60. Quoziente e resto

Soluzione:

```
caramelle = 47
bambini = 5
ciascuno = caramelle // bambini
avanzo = caramelle % bambini
print(f"Ciascuno: {ciascuno}, avanzano: {avanzo}")
```

Output atteso:

```
Ciascuno: 9, avanzano: 2
```

Perché funziona: // distribuisce in parti intere, % dice quante restano fuori.

Errore comune: Usare / che darebbe un decimale, non adatto a contare caramelle intere.

Variante da provare: Chiedi numero di caramelle e bambini all'utente.

Esercizio 61. Iniziali maiuscole

Soluzione:

```
nome = "mario rossi"
print(nome.title())
```

Output atteso:

```
Mario Rossi
```

Perché funziona: .title() rende maiuscola la prima lettera di ogni parola.

Errore comune: Usare .upper() che renderebbe TUTTO maiuscolo.

Variante da provare: Gestisci anche un nome con più di due parole.

Esercizio 62. Prezzo unitario

Soluzione:

```
prezzo = 4.50
bottiglie = 6
unitario = prezzo / bottiglie
print(f"Prezzo per bottiglia: {unitario:.2f} euro")
```

Output atteso:

```
Prezzo per bottiglia: 0.75 euro
```

Perché funziona: Una divisione semplice, utile per confrontare la convenienza di confezioni diverse.

Errore comune: Dimenticare :.2f e mostrare troppi decimali.

Variante da provare: Confronta due confezioni diverse e stampa quale conviene.

Esercizio 63. Maggiore tra tre numeri

Soluzione:

```
a, b, c = 12, 27, 19
if a >= b and a >= c:
    maggiore = a
elif b >= a and b >= c:
    maggiore = b
else:
    maggiore = c
print(f"Il maggiore è: {maggiore}")
```


Output atteso:

```
Il maggiore è: 27
```

Perché funziona: Ogni ramo controlla se un numero è maggiore o uguale agli altri due. L'else cattura il terzo caso.

Errore comune: Dimenticare un confronto (solo $a \geq b$) classificherebbe male in alcuni casi.

Variante da provare: Trova anche il minore tra i tre.

Esercizio 64. Ripeti un messaggio

Soluzione:

```
for i in range(1, 6):  
    print(f"{i}. Python")
```

Output atteso:

```
1. Python  
2. Python  
3. Python  
4. Python  
5. Python
```

Perché funziona: range(1, 6) dà i numeri da 1 a 5, usati per numerare ogni riga.

Errore comune: Usare range(5) e poi stampare i: partirebbe da 0.

Variante da provare: Stampa la parola un numero di volte deciso dall'utente.

Soluzioni - Intermedio

Esercizio 65. Conta fino a 10

Soluzione:

```
for i in range(1, 11):  
    print(i)
```

Output atteso:

```
1  
2  
...  
10
```

Perché funziona: range(1, 11) arriva fino a 10 (l'ultimo numero è escluso). Il for ripete il blocco per ogni numero.

Errore comune: Scrivere range(1, 10): si ferma a 9, perché l'estremo destro è escluso.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 66. Tabellina

Soluzione:

```
n = int(input("Numero: "))  
for i in range(1, 11):  
    print(f"{n} x {i} = {n * i}")
```

Output atteso:

```
(se 7) 7 14 21 ... 70
```

Perché funziona: Dentro il for, i cambia a ogni giro: lo usi per costruire la moltiplicazione.

Errore comune: Dimenticare di usare i nel calcolo: stamperebbe sempre lo stesso risultato.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 67. Somma dei primi N

Soluzione:

```
n = int(input("N: "))  
totale = 0  
for i in range(1, n + 1):  
    totale += i  
print(f"La somma è: {totale}")
```

Output atteso:

```
(se 5) La somma è: 15
```

Perché funziona: L'accumulatore (totale) parte da 0 e cresce a ogni giro: è lo schema più importante dei cicli.

Errore comune: Mettere totale = 0 dentro il for: si azzererebbe a ogni giro, dando sempre l'ultimo numero.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 68. Solo i pari

Soluzione:

```
for i in range(2, 21, 2):  
    print(i)
```

Output atteso:

```
2 4 6 ... 20
```

Perché funziona: range(2, 21, 2) parte da 2 e salta di 2 in 2: il terzo numero è il passo.

Errore comune: Usare range(1, 21, 2) parte da 1 e dà i dispari, non i pari.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 69. Conto alla rovescia

Soluzione:

```
for i in range(5, 0, -1):  
    print(i)  
print("Via!")
```

Output atteso:

```
5  
4  
3  
2  
1  
Via!
```

Perché funziona: range(5, 0, -1) parte da 5 e scende di 1 fino a 1 (lo 0 è escluso). Il passo -1 fa contare all'indietro.

Errore comune: Scrivere range(5, 1, -1): si ferma a 2, perché l'estremo 1 è escluso.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 70. Indovina con while

Soluzione:

```
segreto = 7  
tentativo = int(input("Indovina: "))  
while tentativo != segreto:  
    print("Sbagliato, riprova")  
    tentativo = int(input("Indovina: "))  
print("Esatto!")
```

Output atteso:

```
(quando indovina) Esatto!
```

Perché funziona: Il while ripete finché la condizione resta vera. Va aggiornata la variabile dentro il ciclo, o non finisce mai.

Errore comune: Dimenticare di rifare input() dentro il while: il ciclo diventa infinito.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 71. Media di numeri inseriti

Soluzione:

```
somma = 0  
for i in range(5):  
    n = float(input(f"Numero {i + 1}: "))  
    somma += n  
print(f"Media: {somma / 5:.2f}")
```

Output atteso:

```
(media dei 5)
```

Perché funziona: Combini ciclo, input e accumulatore: schema tipico per elaborare più dati.

Errore comune: Dividere dentro il for invece che dopo: calcoleresti medie parziali sbagliate.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 72. Prima lista

Soluzione:

```
frutti = ["mela", "banana", "kiwi"]  
print(frutti)
```

```
print(frutti[0])
print(frutti[-1])
```

Output atteso:

```
['mela', 'banana', 'kiwi']
mela
kiwi
```

Perché funziona: Le liste si indicizzano da 0. frutti[-1] è una scorciatoia per l'ultimo elemento.

Errore comune: Pensare che il primo elemento sia frutti[1]: no, è frutti[0].

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 73. Aggiungere e togliere

Soluzione:

```
colori = ["rosso", "blu"]
colori.append("verde")
print(colori)
colori.remove("rosso")
print(colori)
```

Output atteso:

```
['rosso', 'blu', 'verde'] poi ['blu', 'verde']
```

Perché funziona: append() aggiunge in fondo, remove() toglie il primo elemento uguale a quello indicato.

Errore comune: remove() di un elemento che non c'è dà errore: controlla sempre che esista.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 74. Scorrere una lista

Soluzione:

```
nomi = ["Anna", "Luca", "Sara", "Marco"]
for nome in nomi:
    print(nome)
```

Output atteso:

```
Anna
Luca
...
```

Perché funziona: Il for su una lista assegna a turno ogni elemento alla variabile: non servono indici.

Errore comune: Usare range(len(nomi)) e nomi[i] funziona ma è più complicato del necessario.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 75. Numerare gli elementi

Soluzione:

```
spesa = ["pane", "latte", "uova"]
for i, prodotto in enumerate(spesa, 1):
    print(f"{i}. {prodotto}")
```

Output atteso:

```
1. pane
2. latte
```

Perché funziona: enumerate(lista, 1) restituisce coppie (numero, elemento) partendo da 1: comodo per liste numerate.

Errore comune: Senza il secondo argomento, enumerate parte da 0.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 76. Massimo e minimo

Soluzione:

```
numeri = [10, 25, 7, 43, 18]
print(max(numeri))
print(min(numeri))
print(sum(numeri))
```

Output atteso:

```
massimo, minimo, somma
```

Perché funziona: max(), min() e sum() lavorano direttamente su una lista di numeri: niente cicli a mano.

Errore comune: Scrivere max(10, 25, ...) elemento per elemento: inutile, basta passare la lista.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 77. Primo dizionario

Soluzione:

```
persona = {"nome": "Anna", "eta": 30, "citta": "Roma"}
print(persona["nome"])
print(persona["eta"])
print(persona["citta"])
```

Output atteso:

```
Anna
30
Roma
```

Perché funziona: Il dizionario associa chiavi a valori. Si accede al valore mettendo la chiave tra parentesi quadre.

Errore comune: Usare le parentesi tonde invece delle quadre per accedere: dà errore.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 78. Scorrere un dizionario

Soluzione:

```
persona = {"nome": "Anna", "eta": 30, "citta": "Roma"}
for chiave, valore in persona.items():
    print(f"{chiave}: {valore}")
```

Output atteso:

```
nome: Anna
eta: 30
citta: Roma
```

Perché funziona: `.items()` restituisce coppie chiave-valore, perfette da scorrere con un `for` a due variabili.

Errore comune: Scorrere il dizionario senza `.items()` dà solo le chiavi, non i valori.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 79. Chiave sicura

Soluzione:

```
persona = {"nome": "Anna", "eta": 30}
print(persona.get("email"))
print(persona.get("email", "Non disponibile"))
```

Output atteso:

```
None oppure Non disponibile
```

Perché funziona: `.get()` restituisce `None` (o un valore di default) se la chiave non c'è, invece di dare errore.

Errore comune: Usare `persona['email']` su una chiave inesistente fa crashare il programma con `KeyError`.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 80. Prima funzione

Soluzione:

```
def saluta():
    print("Ciao!")

saluta()
saluta()
```

Output atteso:

```
Ciao!
Ciao!
```

Perché funziona: `def` crea la funzione, ma il codice dentro viene eseguito solo quando la chiami con `saluta()`.

Errore comune: Definire la funzione e non chiamarla mai: non succede nulla.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 81. Funzione con parametro

Soluzione:

```
def saluta_persona(nome):
    print(f"Ciao, {nome}!")

saluta_persona("Anna")
saluta_persona("Luca")
```

Output atteso:

```
Ciao, Anna!  
Ciao, Luca!
```

Perché funziona: Il parametro (nome) è una variabile che riceve il valore che passi quando chiami la funzione.

Errore comune: Chiamare `saluta_persona()` senza argomento dà errore: il parametro nome è obbligatorio.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 82. Funzione che restituisce

Soluzione:

```
def area Rettangolo(base, altezza):  
    return base * altezza  
  
risultato = area_Rettangolo(5, 3)  
print(risultato)
```

Output atteso:

```
(per 5 e 3) 15
```

Perché funziona: `return` manda indietro un valore che puoi salvare in una variabile e usare. Diverso da `print()`, che solo mostra.

Errore comune: Usare `print()` dentro la funzione invece di `return`: non puoi riutilizzare il risultato in altri calcoli.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 83. Valore di default

Soluzione:

```
def prezzo_con_iva(prezzo, iva=0.22):  
    return prezzo * (1 + iva)  
  
print(prezzo_con_iva(100))  
print(prezzo_con_iva(100, 0.10))
```

Output atteso:

```
122.0 e 110.0
```

Perché funziona: Un parametro con valore di default (`iva=0.22`) può essere omesso nella chiamata: si usa il default.

Errore comune: Mettere il parametro con default prima di uno senza default dà errore di sintassi.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 84. Funzione con più return

Soluzione:

```
def classifica(voto):  
    if voto >= 6:  
        return "Promosso"  
    else:  
        return "Bocciato"  
  
print(classifica(7))  
print(classifica(4))
```

Output atteso:

```
Promosso e Bocciato
```

Perché funziona: Una funzione può avere più `return` in rami diversi: appena ne incontra uno, esce.

Errore comune: Dimenticare il `return` in un ramo: la funzione restituirebbe `None` in quel caso.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 85. Gestire input non valido

Soluzione:

```
try:  
    n = int(input("Numero: "))  
    print(f"Hai scritto {n}")  
except ValueError:  
    print("Non è un numero valido")
```

Output atteso:

```
(se scrivi ciao) Non è un numero valido
```

Perché funziona: try/except cattura l'errore: se int() fallisce, invece di crashare esegue il blocco except.

Errore comune: Mettere troppo codice dentro try: meglio racchiudere solo la riga che può fallire.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 86. Divisione protetta

Soluzione:

```
try:
    a = float(input("Primo: "))
    b = float(input("Secondo: "))
    print(a / b)
except ZeroDivisionError:
    print("Non si può dividere per zero")
```

Output atteso:

```
(se 10 e 0) Non si può dividere per zero
```

Perché funziona: Puoi catturare un errore specifico (ZeroDivisionError) per dare un messaggio chiaro.

Errore comune: Usare un except generico nasconde anche altri errori che vorresti vedere.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 87. Leggere un errore

Soluzione:

```
# NameError: la variabile non è stata definita prima di usarla
# print(eta) # eseguila per vedere l'errore, poi commenta
eta = 30
print(eta)
```

Output atteso:

```
NameError
```

Perché funziona: NameError significa che hai usato un nome che Python non conosce: di solito una variabile non ancora creata o scritta male.

Errore comune: Ignorare il tipo di errore: è proprio quello che ti dice cosa correggere.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 88. Tabella delle tabelline

Soluzione:

```
for n in range(1, 6):
    riga = ""
    for i in range(1, 11):
        riga += f"{n*i:>4}"
    print(f"Tabellina {n}: {riga}")
```

Output atteso:

```
Tabellina 1:    1    2    3 ...
... fino a Tabellina 5
```

Perché funziona: Il ciclo esterno scorre le tabelline (1-5), quello interno i moltiplicatori (1-10). Costruisci la riga e la stampi.

Errore comune: Mettere la print() dentro il ciclo interno: stamperebbe ogni numero su una riga diversa invece che in fila.

Variante da provare: Stampa solo le righe delle tabelline pari (2 e 4).

Esercizio 89. FizzBuzz

Soluzione:

```
for n in range(1, 21):
    if n % 3 == 0 and n % 5 == 0:
        print("FizzBuzz")
    elif n % 3 == 0:
        print("Fizz")
    elif n % 5 == 0:
        print("Buzz")
    else:
        print(n)
```

Output atteso:

```
1, 2, Fizz, 4, Buzz, Fizz, 7, ... 14, FizzBuzz, 16, ...
```

Perché funziona: L'ordine conta: il caso più specifico (entrambi) va controllato per primo, altrimenti non verrebbe mai raggiunto.

Errore comune: Controllare Fizz e Buzz prima di FizzBuzz: il numero 15 stamperebbe solo 'Fizz' e non 'FizzBuzz'.

Variante da provare: Cambia i numeri: Fizz per i multipli di 2, Buzz per quelli di 7.

Esercizio 90. Somma solo dei numeri positivi

Soluzione:

```
numeri = [10, -5, 3, -8, 12, -1]
totale = 0
for n in numeri:
    if n > 0:
        totale += n
print(f"Somma dei positivi: {totale}")
```

Output atteso:

```
Somma dei positivi: 25
```

Perché funziona: L'if dentro il ciclo filtra: solo i numeri positivi entrano nell'accumulatore.

Errore comune: Mettere totale = 0 dentro il ciclo lo azzererebbe a ogni giro.

Variante da provare: Conta anche quanti erano i numeri negativi.

Esercizio 91. Trova il numero più grande

Soluzione:

```
numeri = [23, 45, 12, 67, 34]
massimo = numeri[0]
for n in numeri:
    if n > massimo:
        massimo = n
print(f"Il più grande è: {massimo}")
```

Output atteso:

```
Il più grande è: 67
```

Perché funziona: Inizializzi massimo col primo elemento, poi a ogni giro lo aggiorni se trovi qualcosa di più grande. È come funziona max() sotto.

Errore comune: Inizializzare massimo = 0: se tutti i numeri fossero negativi, il risultato sarebbe sbagliato.

Variante da provare: Trova il minimo con la stessa logica, oppure trova la posizione (indice) del massimo.

Esercizio 92. Indovina il numero con tentativi

Soluzione:

```
segreto = 42
tentativi = 0
trovato = False
while not trovato:
    n = int(input("Indovina (1-100): "))
    tentativi += 1
    if n == segreto:
        trovato = True
        print(f"Bravo! Indovinato in {tentativi} tentativi")
    elif n < segreto:
        print("Più alto")
    else:
        print("Più basso")
```

Output atteso:

```
(interattivo) Più alto / Più basso / Bravo! Indovinato in N tentativi
```

Perché funziona: Il while gira finché trovato resta False. Gli indizi guidano l'utente. Il contatore conta i tentativi.

Errore comune: Dimenticare di rifare input() dentro il while o di aggiornare 'trovato': il ciclo diventa infinito.

Variante da provare: Limita i tentativi a 7: se li supera, l'utente perde.

Esercizio 93. Rimuovi i doppi

Soluzione:

```
originale = ["mela", "pera", "mela", "kiwi", "pera"]
senza_doppi = []
for frutto in originale:
```

```
if frutto not in senza_doppioni:
    senza_doppioni.append(frutto)
print(senza_doppioni)
```

Output atteso:

```
['mela', 'pera', 'kiwi']
```

Perché funziona: 'not in' controlla se l'elemento manca ancora. Lo aggiungi solo in quel caso, così eviti i doppioni.

Errore comune: Usare set() risolverebbe ma perderebbe l'ordine: per mantenerlo serve questo metodo.

Variante da provare: Conta quante volte appariva ogni frutto nella lista originale, usando un dizionario.

Esercizio 94. Inventario con dizionario

Soluzione:

```
magazzino = {"penne": 20, "quaderni": 15, "gomme": 8}
magazzino["penne"] += 5
for prodotto, quantita in magazzino.items():
    print(f"{prodotto}: {quantita}")
```

Output atteso:

```
penne: 25
quaderni: 15
gomme: 8
```

Perché funziona: .items() scorre coppie chiave-valore. L'aggiornamento += somma al valore esistente.

Errore comune: Accedere a una chiave inesistente con []: darebbe KeyError. Usa il nome esatto della chiave.

Variante da provare: Aggiungi un nuovo prodotto al magazzino e segnala quelli con quantità sotto 10.

Esercizio 95. Frequenza delle parole

Soluzione:

```
frase = "il gatto e il cane e il topo"
parole = frase.split()
conteggio = {}
for parola in parole:
    conteggio[parola] = conteggio.get(parola, 0) + 1
for parola, n in conteggio.items():
    print(f"{parola}: {n}")
```

Output atteso:

```
il: 3
gatto: 1
e: 2
cane: 1
topo: 1
```

Perché funziona: .split() divide la frase in parole. .get(parola, 0) restituisce il conteggio attuale o 0 se nuova, poi aggiungi 1.

Errore comune: Usare conteggio[parola] += 1 senza .get(): darebbe KeyError alla prima parola nuova.

Variante da provare: Trova la parola più frequente e stampala.

Esercizio 96. Rubrica telefonica

Soluzione:

```
rubrica = [
    {"nome": "Anna", "tel": "333111"},
    {"nome": "Marco", "tel": "333222"},
    {"nome": "Sara", "tel": "333333"},
]
cercato = "Marco"
trovato = False
for c in rubrica:
    if c["nome"] == cercato:
        print(f"{cercato}: {c['tel']}")
        trovato = True
if not trovato:
    print("Contatto non trovato")
```

Output atteso:

```
Marco: 333222
```


Perché funziona: Scorri la lista di dizionari. Quando il nome coincide, stampi il telefono. Il flag 'trovato' gestisce il caso assente.

Errore comune: Dimenticare il caso 'non trovato': il programma non direbbe nulla se il nome non c'è.

Variante da provare: Permetti la ricerca anche se l'utente scrive il nome minuscolo (usa .lower()).

Esercizio 97. Funzione è primo

Soluzione:

```
def e_primo(n):
    if n < 2:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

for numero in [2, 7, 9, 11, 15]:
    print(f"{numero}: {e_primo(numero)}")
```

Output atteso:

```
2: True
7: True
9: False
11: True
15: False
```

Perché funziona: Se trovi un divisore, restituisci subito False. Se il ciclo finisce senza trovarne, è primo: return True.

Errore comune: Mettere return True dentro il ciclo: restituirebbe troppo presto, al primo numero non divisore.

Variante da provare: Ottimizza controllando i divisori solo fino alla radice quadrata di n.

Esercizio 98. Calcolatrice a funzioni

Soluzione:

```
def somma(a, b): return a + b
def sottrai(a, b): return a - b
def moltiplica(a, b): return a * b
def dividi(a, b):
    if b == 0:
        return "Errore: divisione per zero"
    return a / b

op = ""
a, b = 6, 3
if op == "+": print(somma(a, b))
elif op == "-": print(sottrai(a, b))
elif op == "*": print(moltiplica(a, b))
elif op == "/": print(dividi(a, b))
```

Output atteso:

```
18
```

Perché funziona: Ogni operazione è una funzione separata e riutilizzabile. La divisione gestisce il caso speciale dello zero.

Errore comune: Non gestire la divisione per zero: darebbe ZeroDivisionError.

Variante da provare: Chiedi operazione e numeri all'utente con input() e ripeti finché non sceglie 'esci'.

Esercizio 99. Funzione conta caratteri

Soluzione:

```
def analizza(testo):
    caratteri = len(testo)
    parole = len(testo.split())
    vocali = sum(1 for c in testo.lower() if c in "aeiou")
    return caratteri, parole, vocali

c, p, v = analizza("Python e bello")
print(f"Caratteri: {c}, Parole: {p}, Vocali: {v}")
```

Output atteso:

```
Caratteri: 14, Parole: 3, Vocali: 5
```

Perché funziona: Una funzione può restituire più valori con la virgola. Li ricevi assegnandoli a più variabili.

Errore comune: Ricevere il risultato in una sola variabile: otterresti una tupla, non i tre valori separati.

Variante da provare: Aggiungi il conteggio delle consonanti e degli spazi.

Esercizio 100. Somma cifre di un numero

Soluzione:

```
numero = 1234
somma = 0
n = numero
while n > 0:
    somma += n % 10
    n = n // 10
print(f"Somma delle cifre di {numero}: {somma}")
```

Output atteso:

```
Somma delle cifre di 1234: 10
```

Perché funziona: $n \% 10$ dà l'ultima cifra, $n // 10$ la rimuove. Il while continua finché restano cifre.

Errore comune: Modificare 'numero' invece di una copia 'n': perderesti il valore originale per la stampa.

Variante da provare: Conta anche quante cifre ha il numero.

Esercizio 101. Lista della spesa interattiva

Soluzione:

```
spesa = []
while True:
    p = input("Prodotto (o 'fine'): ")
    if p == "fine":
        break
    spesa.append(p)
print("La tua spesa:")
for i, prod in enumerate(spesa, 1):
    print(f"{i}. {prod}")
```

Output atteso:

```
(interattivo) elenco numerato dei prodotti inseriti
```

Perché funziona: while True con break è lo schema per ripetere finché una condizione di uscita. enumerate numera la lista.

Errore comune: Dimenticare il break: il ciclo continuerebbe all'infinito anche scrivendo 'fine'.

Variante da provare: Permetti anche di rimuovere un prodotto scrivendo 'togli <nome>'.

Esercizio 102. Media senza i voti più bassi

Soluzione:

```
voti = [28, 18, 30, 25, 22]
voti_ordinati = sorted(voti)
rimanenti = voti_ordinati[1:]
media = sum(rimanenti) / len(rimanenti)
print(f"Media senza il più basso: {media:.2f}")
```

Output atteso:

```
Media senza il più basso: 26.25
```

Perché funziona: sorted() ordina dal più piccolo. [1:] salta il primo elemento. Poi media sui rimanenti.

Errore comune: Usare .sort() che modifica la lista originale, se ti serviva intatta: sorted() ne crea una nuova.

Variante da provare: Escludi sia il voto più alto sia il più basso.

Esercizio 103. Conta parole lunghe

Soluzione:

```
parole = ["casa", "computer", "sole", "programmare", "il", "python"]
lunghe = []
for p in parole:
    if len(p) > 5:
        lunghe.append(p)
print(f"Parole lunghe ({len(lunghe)}): {lunghe}")
```

Output atteso:

```
Parole lunghe (3): ['computer', 'programmare', 'python']
```

Perché funziona: len(p) misura ogni parola. Solo quelle sopra 5 caratteri entrano nella nuova lista.

Errore comune: Confondere > con >=: 'python' ha 6 caratteri, rientra; una parola di 5 esatti no.

Variante da provare: Trova la parola più lunga della lista.

Esercizio 104. Fattoriale di un numero

Soluzione:

```
n = 5
fattoriale = 1
for i in range(1, n + 1):
    fattoriale *= i
print(f"{n}! = {fattoriale}")
```

Output atteso:

```
5! = 120
```

Perché funziona: L'accumulatore parte da 1 (non da 0, o moltiplicheresti sempre per zero) e si moltiplica a ogni giro.

Errore comune: Inizializzare fattoriale = 0: qualsiasi moltiplicazione darebbe 0.

Variante da provare: Calcola il fattoriale solo se il numero è positivo, altrimenti stampa un avviso.

Esercizio 105. Sequenza di Fibonacci

Soluzione:

```
a, b = 0, 1
for _ in range(10):
    print(a, end=" ")
    a, b = b, a + b
print()
```

Output atteso:

```
0 1 1 2 3 5 8 13 21 34
```

Perché funziona: a, b = b, a+b aggiorna entrambe le variabili contemporaneamente. end=' ' stampa sulla stessa riga.

Errore comune: Aggiornare a e b in due righe separate: il secondo calcolo userebbe il valore già modificato.

Variante da provare: Stampa solo i numeri di Fibonacci pari.

Esercizio 106. Inverti una lista senza reverse()

Soluzione:

```
originale = [1, 2, 3, 4, 5]
invertita = []
for elemento in originale:
    invertita.insert(0, elemento)
print(invertita)
```

Output atteso:

```
[5, 4, 3, 2, 1]
```

Perché funziona: insert(0, elemento) mette ogni elemento in prima posizione: così l'ultimo inserito (l'ultimo dell'originale) finisce davanti... in realtà il primo finisce in fondo, invertendo la lista.

Errore comune: Usare append() invece di insert(0, ...): otterresti la lista nello stesso ordine.

Variante da provare: Verifica se la lista è palindroma confrontandola con la sua versione invertita.

Esercizio 107. Dividi pari e dispari

Soluzione:

```
numeri = [4, 7, 2, 9, 6, 1, 8]
pari = []
dispari = []
for n in numeri:
    if n % 2 == 0:
        pari.append(n)
    else:
        dispari.append(n)
print(f"Pari: {pari}")
print(f"Dispari: {dispari}")
```

Output atteso:

```
Pari: [4, 2, 6, 8]
Dispari: [7, 9, 1]
```

Perché funziona: L'if con il modulo decide in quale lista mettere ogni numero. Due liste inizializzate prima del ciclo.

Errore comune: Inizializzare le liste dentro il ciclo: si svuoterebbero a ogni giro.

Variante da provare: Conta anche somma dei pari e somma dei dispari.

Esercizio 108. Cerca in un dizionario annidato

Soluzione:

```
studenti = {  
    "Anna": {"voto": 28, "citta": "Roma"},  
    "Marco": {"voto": 22, "citta": "Milano"},  
}  
print(f"Voto di Anna: {studenti['Anna']['voto']}")  
print(f"Città di Marco: {studenti['Marco']['citta']}")
```

Output atteso:

```
Voto di Anna: 28  
Città di Marco: Milano
```

Perché funziona: studenti['Anna'] dà il dizionario interno; aggiungendo ['voto'] accedi al valore annidato.

Errore comune: Dimenticare un livello di parentesi quadre quando accedi a un dizionario dentro un altro.

Variante da provare: Scorri tutti gli studenti e stampa nome, voto e città di ognuno.

Esercizio 109. Validazione con ciclo

Soluzione:

```
eta = -1  
while eta < 0 or eta > 120:  
    eta = int(input("Inserisci un'età valida (0-120): "))  
print(f"Età accettata: {eta}")
```

Output atteso:

```
(interattivo) Età accettata: <valore valido>
```

Perché funziona: Il while ripete finché l'età è fuori intervallo. Si esce solo con un valore valido.

Errore comune: Inizializzare eta con un valore valido: il ciclo non partirebbe nemmeno una volta.

Variante da provare: Aggiungi un try/except per gestire il caso in cui l'utente scrive testo invece di un numero.

Esercizio 110. Conta le occorrenze di un carattere

Soluzione:

```
frase = "Amaca con ananas"  
frase = frase.lower()  
conta = 0  
for c in frase:  
    if c == "a":  
        conta += 1  
print(f"La lettera a compare {conta} volte")
```

Output atteso:

```
La lettera a compare 6 volte
```

Perché funziona: .lower() uniforma le maiuscole, così conti sia 'A' sia 'a'. L'accumulatore conta le occorrenze.

Errore comune: Dimenticare .lower(): conteresti solo le 'a' minuscole, saltando la 'A' iniziale.

Variante da provare: Usa il metodo .count('a') e verifica di ottenere lo stesso risultato.

Esercizio 111. Somma di una colonna CSV testuale

Soluzione:

```
righe = ["Anna,100", "Marco,250", "Sara,75"]  
totale = 0  
for riga in righe:  
    nome, importo = riga.split(",")  
    totale += float(importo)  
print(f"Totale: {totale:.2f}")
```

Output atteso:

```
Totale: 425.00
```

Perché funziona: split(',') divide nome e importo. float() converte il testo in numero prima di sommarlo.

Errore comune: Sommare 'importo' senza float(): concateneresti testo invece di sommare numeri.

Variante da provare: Stampa anche il nome con l'importo più alto.

Esercizio 112. Conta maiuscole e minuscole

Soluzione:

```
frase = "Ciao Mondo Python"
mai = 0
min_ = 0
for c in frase:
    if c.isupper():
        mai += 1
    elif c.islower():
        min_ += 1
print(f"Maiuscole: {mai}, minuscole: {min_}")
```

Output atteso:

```
Maiuscole: 3, minuscole: 12
```

Perché funziona: `.isupper()` e `.islower()` dicono se un carattere è maiuscolo o minuscolo. Gli spazi non contano in nessuno dei due.

Errore comune: Contare gli spazi come lettere: i metodi `isupper/islower` li escludono correttamente.

Variante da provare: Conta anche le cifre con `.isdigit()`.

Esercizio 113. Genera una password semplice**Soluzione:**

```
import random
caratteri = "abcdefghijklmnopqrstuvwxyz0123456789"
password = ""
for _ in range(8):
    password += random.choice(caratteri)
print(f"Password generata: {password}")
```

Output atteso:

```
Password generata: (8 caratteri casuali)
```

Perché funziona: `random.choice()` pesca un carattere a caso. Ripetuto 8 volte e concatenato, forma la password.

Errore comune: Dimenticare di importare `random` o di inizializzare `password` come stringa vuota.

Variante da provare: Aggiungi anche lettere maiuscole e un paio di simboli.

Esercizio 114. Media mobile semplice**Soluzione:**

```
valori = [10, 20, 30, 40]
for i in range(len(valori) - 1):
    media = (valori[i] + valori[i+1]) / 2
    print(f"Media tra {valori[i]} e {valori[i+1]}: {media}")
```

Output atteso:

```
Media tra 10 e 20: 15.0
Media tra 20 e 30: 25.0
Media tra 30 e 40: 35.0
```

Perché funziona: `range(len-1)` evita di uscire dalla lista quando accedi a `i+1`. Calcoli la media di ogni coppia.

Errore comune: Usare `range(len(valori))`: all'ultimo giro `valori[i+1]` sarebbe fuori dalla lista (`IndexError`).

Variante da provare: Calcola la media mobile su gruppi di 3 valori.

Esercizio 115. Verifica parentesi bilanciate**Soluzione:**

```
testo = "(a + (b * c))"
bilancio = 0
valida = True
for c in testo:
    if c == "(":
        bilancio += 1
    elif c == ")":
        bilancio -= 1
    if bilancio < 0:
        valida = False
        break
if bilancio != 0:
    valida = False
print("Bilanciate" if valida else "Non bilanciate")
```

Output atteso:

Bilanciate

Perché funziona: Il contatore sale con '(' e scende con ')'. Se va sotto zero o non torna a zero alla fine, non sono bilanciate.

Errore comune: Controllare solo il totale finale senza verificare che non scenda mai sotto zero: ')' passerebbe per errore.

Variante da provare: Prova con una stringa sbilanciata come '(a))' e verifica che lo rilevi.

Esercizio 116. Triangolo di asterischi

Soluzione:

```
for riga in range(1, 6):  
    print("*" * riga)
```

Output atteso:

```
*  
**  
***  
****  
*****
```

Perché funziona: '*' moltiplicato per il numero di riga produce il numero giusto di asterischi. range(1,6) dà le righe da 1 a 5.

Errore comune: Usare un ciclo annidato manuale quando '*' * riga è molto più semplice.

Variante da provare: Stampa il triangolo al contrario (5 asterischi in cima, 1 in fondo).

Esercizio 117. Cifrario di Cesare semplice

Soluzione:

```
parola = "ciao"  
cifrata = ""  
for c in parola:  
    cifrata += chr(ord(c) + 1)  
print(f"{parola} -> {cifrata}")
```

Output atteso:

ciao -> djbp

Perché funziona: ord(c)+1 sposta il codice del carattere di uno; chr() lo riconverte in lettera. È il principio del cifrario di Cesare.

Errore comune: Non gestire la 'z' che andrebbe oltre l'alfabeto: in questa versione semplice diventa '{'.

Variante da provare: Decifra il messaggio spostando di -1, e gestisci il ritorno della 'z' alla 'a'.

Soluzioni - Avanzato

Esercizio 118. Statistiche su una lista di voti

Soluzione:

```
voti = [28, 30, 26, 24, 30, 27]  
print(f"Numero voti: {len(voti)}")  
print(f"Media: {sum(voti) / len(voti):.2f}")  
print(f"Massimo: {max(voti)}")  
print(f"Minimo: {min(voti)}")
```

Output atteso:

6 voti, media, max, min

Perché funziona: Con le funzioni base sulle liste fai già una piccola analisi dei dati: è il primo passo verso la data analysis.

Errore comune: Dividere sum(voti) per un numero fisso invece che per len(voti): sbaglia se la lista cambia lunghezza.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 119. Contare con un criterio

Soluzione:

```
voti = [28, 30, 26, 24, 30, 27]  
contatore = 0  
for v in voti:  
    if v >= 28:  
        contatore += 1
```

```
print(f"Voti almeno 28: {contatore}")
```

Output atteso:

```
3 voti >= 28
```

Perché funziona: Contare con un criterio (filtrare e contare) è un'operazione fondamentale nell'analisi dei dati.

Errore comune: Aumentare il contatore fuori dall'if: conteresti tutti i voti, non solo quelli ≥ 28 .

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 120. Dal testo ai dati

Soluzione:

```
riga = "Anna;30;Roma"
pezzi = riga.split(";")
print(pezzi[0])
print(pezzi[1])
print(pezzi[2])
```

Output atteso:

```
Anna 30 Roma
```

Perché funziona: `.split(';')` divide una stringa in una lista usando il punto e virgola come separatore: è così che si leggono i file CSV.

Errore comune: Dimenticare che `split` restituisce una lista di stringhe: `pezzi[1]` è "30" (testo), non il numero 30.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 121. Mini analisi di un dataset

Soluzione:

```
prodotti = [
    {"nome": "Penna", "prezzo": 1.50},
    {"nome": "Quaderno", "prezzo": 3.00},
    {"nome": "Zaino", "prezzo": 25.00},
]
totale = 0
for p in prodotti:
    print(f"{p['nome']}: {p['prezzo']:.2f} euro")
    totale += p["prezzo"]
print(f"Totale: {totale:.2f} euro")
```

Output atteso:

```
elenco + Totale
```

Perché funziona: Una lista di dizionari è il modo più comune di rappresentare una tabella di dati in Python: ogni dizionario è una riga.

Errore comune: Accedere a `p['prezzo']` con la chiave sbagliata (es. 'prezzi') dà `KeyError`.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 122. Usare l'AI come tutor (esercizio guidato)

Soluzione:

```
# Esempio di prompt efficace da dare all'AI:
# "Sono un principiante assoluto di Python. Spiegami questo codice
# riga per riga, con parole semplici, senza dare per scontato nulla.
# Poi dimmi un possibile errore tipico. Ecco il codice: <incolla>"
```

Output atteso:

```
(dipende dai dati inseriti)
```

Perché funziona: Saper chiedere all'AI è una competenza: un prompt specifico, che dichiara il tuo livello, ottiene spiegazioni molto migliori.

Errore comune: Chiedere solo 'spiega questo codice' senza dire il livello: rischi una risposta troppo tecnica.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 123. Riconoscere un errore dell'AI

Soluzione:

```
# Il codice corretto è:
voti = [28, 30, 26]
media = sum(voti) / len(voti) # era len(voto): variabile sbagliata
print(media)
```

Output atteso:

```
len(voto) dovrebbe essere len(voti)
```

Perché funziona: Anche l'AI sbaglia: qui ha scritto `len(voto)` invece di `len(voti)`. Saper leggere il codice ti permette di accorgertene.

Errore comune: Fidarsi del codice dell'AI senza provarlo: questo errore darebbe `NameError` perché `voto` non esiste.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 124. Il tuo primo report

Soluzione:

```
spese = [12.50, 8.00, 25.30, 4.20, 15.00]
print("=== REPORT SPESE ===")
print(f"Numero spese: {len(spese)}")
print(f"Totale: {sum(spese):.2f} euro")
print(f"Media: {sum(spese) / len(spese):.2f} euro")
print(f"Spesa massima: {max(spese):.2f} euro")
```

Output atteso:

```
report completo
```

Perché funziona: Questo è un vero mini-report: è esattamente il tipo di output che produce un'analisi dati di base.

Errore comune: Dimenticare `:.2f` sui soldi: i centesimi verrebbero mostrati con tanti decimali poco professionali.

Variante da provare: Prova a cambiare i valori di partenza e verifica che il risultato cambi di conseguenza.

Esercizio 125. Media voti da input multipli

Soluzione:

```
quanti = int(input("Quanti voti? "))
voti = []
for i in range(quanti):
    v = float(input(f"Voto {i+1}: "))
    voti.append(v)
if voti:
    print(f"Media: {sum(voti)/len(voti):.2f}")
    print(f"Massimo: {max(voti)}")
    print(f"Minimo: {min(voti)}")
```

Output atteso:

```
(interattivo) Media: ..., Massimo: ..., Minimo: ...
```

Perché funziona: Costruisci la lista dinamicamente con `append()` dentro il ciclo. Il controllo `'if voti'` evita la divisione per zero se la lista è vuota.

Errore comune: Non controllare se la lista è vuota: con zero voti, `sum/len` darebbe `ZeroDivisionError`.

Variante da provare: Calcola anche quanti voti sono sufficienti (≥ 6) e la loro percentuale.

Esercizio 126. Report spese da dizionari

Soluzione:

```
spese = [
    {"voce": "Affitto", "importo": 600},
    {"voce": "Spesa", "importo": 250},
    {"voce": "Bollette", "importo": 150},
]
totale = sum(s["importo"] for s in spese)
print("=== REPORT SPESE ===")
for s in spese:
    perc = s["importo"] / totale * 100
    print(f"{s['voce']:<10} {s['importo']:>6} euro ({perc:4.1f}%)")
print(f"{'TOTALE':<10} {totale:>6} euro")
```

Output atteso:

```
=== REPORT SPESE ===
Affitto      600 euro (60.0%)
Spesa        250 euro (25.0%)
Bollette     150 euro (15.0%)
TOTALE      1000 euro
```

Perché funziona: Prima il totale con `sum()`. Poi per ogni voce calcoli la percentuale e usi la formattazione allineata per un report ordinato.

Errore comune: Calcolare la percentuale prima di aver calcolato il totale: il totale deve esistere prima.

Variante da provare: Ordina le voci dalla più costosa alla meno costosa prima di stamparle.

Esercizio 127. Analisi di un CSV simulato

Soluzione:

```
dati = "prodotto;quantita;prezzo\nPane;3;1.20\nLatte;2;1.50\nUova;6;0.30"
with open("v.csv", "w", encoding="utf-8") as f:
    f.write(dati)

incasso = 0
max_q = 0
top = ""
with open("v.csv", "r", encoding="utf-8") as f:
    f.readline()
    for riga in f:
        try:
            p, q, pr = riga.strip().split(";")
            q, pr = int(q), float(pr)
            incasso += q * pr
            if q > max_q:
                max_q, top = q, p
        except (ValueError, IndexError):
            continue
print(f"Incasso totale: {incasso:.2f} euro")
print(f"Più venduto: {top} ({max_q} pezzi)")
```

Output atteso:

```
Incasso totale: 8.70 euro
Più venduto: Uova (6 pezzi)
```

Perché funziona: Leggi il CSV come testo, separa i campi con split, converti i numeri. try/except salta righe rotte. Tieni traccia del massimo per il prodotto più venduto.

Errore comune: Dimenticare int()/float(): i campi del CSV sono testo e i calcoli darebbero errore.

Variante da provare: Aggiungi il calcolo dell'incasso per ogni singolo prodotto.

Esercizio 128. Verifica codice suggerito dall'AI

Soluzione:

```
presenze_lista = ["Anna", "Marco", "Anna", "Sara", "Marco", "Anna"]
presenze = {}
for nome in presenze_lista:
    presenze[nome] = presenze.get(nome, 0) + 1
for nome, n in presenze.items():
    print(f"{nome}: {n}")
```

Output atteso:

```
Anna: 3
Marco: 2
Sara: 1
```

Perché funziona: L'errore tipico dell'AI era usare += su una chiave non ancora esistente. La correzione usa .get(nome, 0) per partire da 0.

Errore comune: presenze[nome] += 1 su un dizionario vuoto dà KeyError: la chiave non esiste ancora.

Variante da provare: Scrivi il prompt che useresti per chiedere all'AI di spiegarti perché serve .get().

Esercizio 129. Mini tutor di tabelline

Soluzione:

```
import random
punteggio = 0
for i in range(3):
    a = random.randint(2, 9)
    b = random.randint(2, 9)
    risp=""
    # in esecuzione reale: risp = int(input(f"{a} x {b} = "))
    risp = a * b # simulazione: risposta corretta
    if risp == a * b:
        punteggio += 1
print(f"Punteggio: {punteggio}/3")
```

Output atteso:

```
Punteggio: 3/3
```

Perché funziona: random.randint(2,9) genera numeri casuali. Confronti la risposta col prodotto vero e aggiorni il punteggio. (Qui l'input è simulato per poter eseguire l'esempio.)

Errore comune: Confrontare la risposta come testo: input() dà una stringa, va convertita con int().

Variante da provare: Aggiungi un messaggio finale diverso a seconda del punteggio (es. 'Ottimo!' se 3/3).

Esercizio 130. Raggruppa studenti per esito

Soluzione:

```
studenti = [
    {"nome": "Anna", "voto": 28},
    {"nome": "Marco", "voto": 15},
    {"nome": "Sara", "voto": 22},
    {"nome": "Luca", "voto": 17},
]
gruppi = {"promossi": [], "bocciati": []}
for s in studenti:
    if s["voto"] >= 18:
        gruppi["promossi"].append(s["nome"])
    else:
        gruppi["bocciati"].append(s["nome"])
print(f"Promossi ({len(gruppi['promossi'])}): {gruppi['promossi']}")
print(f"Bocciati ({len(gruppi['bocciati'])}): {gruppi['bocciati']}")
```

Output atteso:

```
Promossi (2): ['Anna', 'Sara']
Bocciati (2): ['Marco', 'Luca']
```

Perché funziona: Un dizionario con due liste raccoglie i nomi smistati dall'if. Così raggruppi e conti insieme.

Errore comune: Inizializzare le liste fuori dal dizionario e dimenticare di aggiungervi i nomi.

Variante da provare: Calcola anche la media voti dei soli promossi.

Esercizio 131. Salva e rileggi una rubrica su file

Soluzione:

```
rubrica = [("Anna", "333111"), ("Marco", "333222")]
with open("rubrica.txt", "w", encoding="utf-8") as f:
    for nome, tel in rubrica:
        f.write(f"{nome},{tel}\n")

print("Rubrica salvata. Rileggo:")
with open("rubrica.txt", "r", encoding="utf-8") as f:
    for riga in f:
        nome, tel = riga.strip().split(",")
        print(f"{nome}: {tel}")
```

Output atteso:

```
Rubrica salvata. Rileggo:
Anna: 333111
Marco: 333222
```

Perché funziona: Scrivi i dati in formato CSV semplice. In lettura, strip() pulisce e split(',') separa i campi.

Errore comune: Dimenticare strip() prima di split: l'a-capo finirebbe attaccato al telefono.

Variante da provare: Aggiungi un nuovo contatto al file senza cancellare quelli esistenti (modalità append).

Esercizio 132. Statistiche di vendita per categoria

Soluzione:

```
vendite = [
    {"cat": "Cibo", "imp": 120},
    {"cat": "Casa", "imp": 80},
    {"cat": "Cibo", "imp": 60},
    {"cat": "Tech", "imp": 240},
]
totali = {}
for v in vendite:
    totali[v["cat"]] = totali.get(v["cat"], 0) + v["imp"]
generale = sum(totali.values())
print("=== INCASSI PER CATEGORIA ===")
for cat, imp in totali.items():
    print(f"{cat}<6} {imp:>4} euro ({imp/generale*100:4.1f}%)")
print(f"{'TOT':<6} {generale:>4} euro")
```

Output atteso:

```
=== INCASSI PER CATEGORIA ===
Cibo      180 euro (36.0%)
Casa      80 euro (16.0%)
Tech      240 euro (48.0%)
TOT       500 euro
```

Perché funziona: Il dizionario 'totali' accumula per categoria con `.get()`. Poi `sum(totali.values())` dà il totale generale per le percentuali.

Errore comune: Calcolare le percentuali prima di aver sommato tutti gli importi nel dizionario.

Variante da provare: Trova e stampa la categoria che ha incassato di più.

Esercizio 133. Bilancio entrate-uscite

Soluzione:

```
movimenti = [
    {"desc": "Stipendio", "imp": 1500},
    {"desc": "Affitto", "imp": -600},
    {"desc": "Spesa", "imp": -200},
    {"desc": "Rimborso", "imp": 50},
]
entrate = sum(m["imp"] for m in movimenti if m["imp"] > 0)
uscite = sum(m["imp"] for m in movimenti if m["imp"] < 0)
saldo = entrate + uscite
print(f"Entrate: {entrate} euro")
print(f"Uscite: {uscite} euro")
print(f"Saldo: {saldo} euro")
```

Output atteso:

```
Entrate: 1550 euro
Uscite: -800 euro
Saldo: 750 euro
```

Perché funziona: `sum()` con un `if` interno filtra per segno. Le uscite sono già negative, quindi il saldo è `entrate + uscite`.

Errore comune: Sottrarre le uscite (già negative): le sommeresti due volte sbagliando il segno.

Variante da provare: Aggiungi il movimento più grande in valore assoluto.

Esercizio 134. Parole uniche in un testo

Soluzione:

```
testo = "Il cane corre. Il gatto dorme, il cane salta!"
testo = testo.lower().replace(".", " ").replace(",", " ").replace("!", " ")
parole = testo.split()
uniche = []
for p in parole:
    if p not in uniche:
        uniche.append(p)
print(f"Parole diverse: {len(uniche)}")
print(uniche)
```

Output atteso:

```
Parole diverse: 6
['il', 'cane', 'corre', 'gatto', 'dorme', 'salta']
```

Perché funziona: Prima pulisci (minuscolo + toglì punteggiatura), poi raccogli solo le parole mai viste. `len()` conta le uniche.

Errore comune: Non rimuovere la punteggiatura: 'cane.' e 'cane' verrebbero contate come diverse.

Variante da provare: Trova la parola che si ripete di più.

Esercizio 135. Tabella ASCII di un testo

Soluzione:

```
parola = "Ciao"
for c in parola:
    print(f"{c} -> {ord(c)}")
```

Output atteso:

```
C -> 67
i -> 105
a -> 97
o -> 111
```

Perché funziona: `ord()` converte un carattere nel suo codice numerico. Utile per capire come il computer rappresenta il testo.

Errore comune: Passare più di un carattere a ord(): vuole un singolo carattere.

Variante da provare: Fai l'inverso con chr(): dato un codice, stampa il carattere corrispondente.

Esercizio 136. Filtra e ordina prodotti

Soluzione:

```
prodotti = [
    {"nome": "Penna", "prezzo": 1.5},
    {"nome": "Zaino", "prezzo": 25},
    {"nome": "Quaderno", "prezzo": 3},
    {"nome": "Astuccio", "prezzo": 8},
]
economici = [p for p in prodotti if p["prezzo"] < 10]
economici = sorted(economici, key=lambda p: p["prezzo"])
for p in economici:
    print(f"{p['nome']}: {p['prezzo']} euro")
```

Output atteso:

```
Penna: 1.5 euro
Quaderno: 3 euro
Astuccio: 8 euro
```

Perché funziona: La list comprehension filtra sotto i 10 euro. sorted con key=lambda ordina per prezzo crescente.

Errore comune: Ordinare prima di filtrare funziona comunque, ma ordinare la lista già filtrata è più efficiente.

Variante da provare: Ordina invece dal più costoso al più economico (reverse=True).

Esercizio 137. Registro presenze settimanale

Soluzione:

```
presenze = {
    "Anna": ["lun", "mar", "mer", "ven"],
    "Marco": ["lun", "mer"],
    "Sara": ["lun", "mar", "mer", "gio", "ven"],
}
for nome, giorni in presenze.items():
    print(f"{nome}: {len(giorni)} giorni")
piu_presente = max(presenze, key=lambda n: len(presenze[n]))
print(f"Più presente: {piu_presente}")
```

Output atteso:

```
Anna: 4 giorni
Marco: 2 giorni
Sara: 5 giorni
Più presente: Sara
```

Perché funziona: Ogni valore è una lista di giorni: len() conta i giorni lavorati. max con key trova chi ne ha di più.

Errore comune: Contare le chiavi invece dei giorni nelle liste: conteresti i dipendenti, non le presenze.

Variante da provare: Trova chi ha lavorato meno giorni e calcola la media di presenze.

Esercizio 138. Mini analisi di un voto medio per materia

Soluzione:

```
voti = [
    {"materia": "Mate", "voto": 7},
    {"materia": "Ita", "voto": 8},
    {"materia": "Mate", "voto": 9},
    {"materia": "Ita", "voto": 6},
    {"materia": "Mate", "voto": 8},
]
somme = {}
conteggi = {}
for v in voti:
    m = v["materia"]
    somme[m] = somme.get(m, 0) + v["voto"]
    conteggi[m] = conteggi.get(m, 0) + 1
for materia in somme:
    media = somme[materia] / conteggi[materia]
    print(f"{materia}: media {media:.2f}")
```

Output atteso:

```
Mate: media 8.00
```

Ita: media 7.00

Perché funziona: Due dizionari paralleli accumulano somma e numero di voti per materia. Alla fine la media è somma/conteggio.

Errore comune: Usare un solo dizionario per la somma e dimenticare il conteggio: non potresti calcolare la media.

Variante da provare: Trova la materia con la media più alta.

Esercizio 139. Promemoria scadenze ordinate

Soluzione:

```
scadenze = [
    {"cosa": "Bolletta", "giorno": 15},
    {"cosa": "Affitto", "giorno": 5},
    {"cosa": "Palestra", "giorno": 28},
]
ordinate = sorted(scadenze, key=lambda s: s["giorno"])
print("Scadenze in ordine:")
for s in ordinate:
    print(f"Giorno {s['giorno']}: {s['cosa']}")
```

Output atteso:

```
Scadenze in ordine:
Giorno 5: Affitto
Giorno 15: Bolletta
Giorno 28: Palestra
```

Perché funziona: sorted con key=lambda s: s['giorno'] ordina i dizionari in base al giorno. La lista originale resta intatta.

Errore comune: Ordinare per descrizione invece che per giorno: otterresti l'ordine alfabetico, non temporale.

Variante da provare: Evidenzia le scadenze entro i prossimi 10 giorni (rispetto a un giorno 'oggi').

Esercizio 140. Top 3 prodotti per vendite

Soluzione:

```
prodotti = [
    {"nome": "A", "venduti": 30},
    {"nome": "B", "venduti": 90},
    {"nome": "C", "venduti": 50},
    {"nome": "D", "venduti": 70},
]
classifica = sorted(prodotti, key=lambda p: p["venduti"], reverse=True)
for p in classifica[:3]:
    print(f"{p['nome']}: {p['venduti']} venduti")
```

Output atteso:

```
B: 90 venduti
D: 70 venduti
C: 50 venduti
```

Perché funziona: sorted con reverse=True ordina dal più venduto. [:3] prende i primi tre della classifica.

Errore comune: Dimenticare reverse=True: otterresti i 3 MENO venduti.

Variante da provare: Calcola anche la percentuale di vendite dei top 3 sul totale.

Esercizio 141. Conta per fascia d'età

Soluzione:

```
eta_lista = [5, 17, 34, 8, 22, 45, 60, 14]
fasce = {"bambini": 0, "ragazzi": 0, "adulti": 0}
for e in eta_lista:
    if e <= 12:
        fasce["bambini"] += 1
    elif e <= 25:
        fasce["ragazzi"] += 1
    else:
        fasce["adulti"] += 1
for fascia, n in fasce.items():
    print(f"{fasce}: {n}")
```

Output atteso:

```
bambini: 2
ragazzi: 3
```

```
adulti: 3
```

Perché funziona: Un dizionario tiene i conteggi. L'if/elif/else smista ogni età nella fascia corretta.

Errore comune: Sovrapporre le fasce (es. ragazzi 12-25 dopo bambini 0-12) conterebbe due volte chi ha 12 anni.

Variante da provare: Calcola la percentuale di ogni fascia sul totale.

Esercizio 142. Pulizia di una lista di email

Soluzione:

```
grezze = [" Anna@Mail.com ", "non valida", "MARCO@test.IT", " "]
pulite = []
for e in grezze:
    e = e.strip().lower()
    if "@" in e:
        pulite.append(e)
print(pulite)
```

Output atteso:

```
['anna@mail.com', 'marco@test.it']
```

Perché funziona: strip() toglie gli spazi, lower() uniforma. Il controllo '@' in è scarta le righe che non sono email.

Errore comune: Non scartare le righe senza @: includeresti dati non validi nella lista pulita.

Variante da provare: Conta quante email sono state scartate perché non valide.

Esercizio 143. Calcolatrice di sconti progressivi

Soluzione:

```
def sconto(totale):
    if totale > 200:
        perc = 15
    elif totale > 100:
        perc = 10
    elif totale > 50:
        perc = 5
    else:
        perc = 0
    return totale * (1 - perc / 100)

for t in [40, 80, 150, 300]:
    print(f"{t} euro -> {sconto(t):.2f} euro")
```

Output atteso:

```
40 euro -> 40.00 euro
80 euro -> 76.00 euro
150 euro -> 135.00 euro
300 euro -> 255.00 euro
```

Perché funziona: Controllando dalla soglia più alta, assegna la percentuale corretta. La funzione restituisce il totale già scontato.

Errore comune: Controllare le soglie dalla più bassa: un importo di 300 prenderebbe subito il 5% invece del 15%.

Variante da provare: Aggiungi una soglia del 20% sopra i 500 euro.

Esercizio 144. Dashboard testuale di un magazzino

Soluzione:

```
magazzino = [
    {"prod": "Penne", "qta": 50},
    {"prod": "Quaderni", "qta": 8},
    {"prod": "Gomme", "qta": 3},
    {"prod": "Righelli", "qta": 25},
]
print("=== MAGAZZINO ===")
bassi = 0
for m in magazzino:
    avviso = " <-- SCORTA BASSA" if m["qta"] < 10 else ""
    if m["qta"] < 10:
        bassi += 1
    print(f"{m['prod']}:<10} {m['qta']:>3}{avviso}")
print(f"\nProdotti da riordinare: {bassi}")
```

Output atteso:

```
=== MAGAZZINO ===
Penne      50
Quaderni    8 <-- SCORTA BASSA
Gomme      3 <-- SCORTA BASSA
Righelli   25

Prodotti da riordinare: 2
```

Perché funziona: L'avviso è una stringa condizionale (vuota se la scorta è ok). Il contatore 'bassi' conta i prodotti da riordinare.

Errore comune: Dimenticare di gestire il caso 'scorta ok' lasciando l'avviso sempre attivo.

Variante da provare: Ordina il report mettendo in cima i prodotti con scorta più bassa.

Esercizio 145. Riepilogo di un sondaggio

Soluzione:

```
risposte = ["si", "no", "si", "forse", "si", "no", "si"]
conteggio = {}
for r in risposte:
    conteggio[r] = conteggio.get(r, 0) + 1
totale = len(risposte)
print("=== RISULTATI SONDAGGIO ===")
for risposta, n in conteggio.items():
    print(f"{risposta:<6} {n} ({n/totale*100:.1f}%)")
```

Output atteso:

```
=== RISULTATI SONDAGGIO ===
si      4 (57.1%)
no      2 (28.6%)
forse   1 (14.3%)
```

Perché funziona: Il dizionario conta ogni tipo di risposta. La percentuale è il conteggio diviso il totale per 100.

Errore comune: Dividere per il numero di tipi di risposta invece che per il totale delle risposte.

Variante da provare: Trova e stampa la risposta più frequente.

Esercizio 146. Conta giorni lavorativi

Soluzione:

```
giorni = ["lun", "mar", "sab", "mer", "dom", "gio", "ven"]
weekend = ["sab", "dom"]
lavorativi = 0
for g in giorni:
    if g not in weekend:
        lavorativi += 1
print(f"Giorni lavorativi: {lavorativi}")
```

Output atteso:

```
Giorni lavorativi: 5
```

Perché funziona: 'not in weekend' è vero per i giorni feriali. Così conti solo quelli, escludendo il weekend.

Errore comune: Confrontare g con 'sab' e 'dom' separatamente con and: 'g != sab and g != dom' funziona ma 'not in' è più pulito.

Variante da provare: Conta separatamente anche i giorni di weekend.

Esercizio 147. Convertitore di valuta con tabella

Soluzione:

```
tassi = {"USD": 1.08, "GBP": 0.85, "JPY": 160.0}
def converti(euro, valuta):
    if valuta not in tassi:
        return "Valuta non disponibile"
    return euro * tassi[valuta]

for v in ["USD", "GBP", "JPY"]:
    print(f"100 euro = {converti(100, v):.2f} {v}")
```

Output atteso:

```
100 euro = 108.00 USD
100 euro = 85.00 GBP
100 euro = 16000.00 JPY
```

Perché funziona: Il dizionario è una tabella di conversione. La funzione cerca il tasso e moltiplica, gestendo le valute assenti.

Errore comune: Non gestire la valuta assente: accedere a `tassi["XYZ"]` inesistente darebbe `KeyError`.

Variante da provare: Aggiungi la conversione inversa: da valuta estera a euro.

Esercizio 148. Analisi temperature settimanali

Soluzione:

```
temp = [22, 25, 19, 28, 30, 24, 21]
media = sum(temp) / len(temp)
massima = max(temp)
minima = min(temp)
sopra_media = 0
for t in temp:
    if t > media:
        sopra_media += 1
print(f"Media: {media:.2f}")
print(f"Massima: {massima}, Minima: {minima}")
print(f"Giorni sopra la media: {sopra_media}")
```

Output atteso:

```
Media: 24.14
Massima: 30, Minima: 19
Giorni sopra la media: 3
```

Perché funziona: Prima calcoli la media con `sum/len`, poi la usi come riferimento per contare i giorni più caldi della media.

Errore comune: Confrontare con la media prima di averla calcolata: la media deve esistere prima del ciclo di conteggio.

Variante da provare: Trova in che giorno (indice) si è registrata la temperatura massima.

Esercizio 149. Sistema di login semplice

Soluzione:

```
utenti = {"anna": "pass123", "marco": "qwerty"}
def login(user, pwd):
    if user in utenti and utenti[user] == pwd:
        return "Accesso consentito"
    return "Credenziali errate"

print(login("anna", "pass123"))
print(login("anna", "sbagliata"))
print(login("ignoto", "x"))
```

Output atteso:

```
Accesso consentito
Credenziali errate
Credenziali errate
```

Perché funziona: Prima controlli che l'utente esista (`user in utenti`), poi che la password coincida. L'`and` garantisce entrambe le condizioni.

Errore comune: Controllare la password senza prima verificare che l'utente esista: `utenti[user]` su un utente assente darebbe `KeyError`.

Variante da provare: Aggiungi un conteggio dei tentativi falliti e blocca dopo 3 errori.

Esercizio 150. Calcola la rata di un prestito

Soluzione:

```
def rata(importo, mesi):
    return importo / mesi

importo = 6000
mesi = 24
print(f"Rata mensile: {rata(importo, mesi):.2f} euro")
print(f"Totale restituito: {rata(importo, mesi) * mesi:.2f} euro")
```

Output atteso:

```
Rata mensile: 250.00 euro
Totale restituito: 6000.00 euro
```

Perché funziona: Una divisione incapsulata in una funzione riutilizzabile. Senza interessi, il totale restituito uguaglia l'importo.

Errore comune: Dividere per zero mesi: gestisci il caso o eviti mesi = 0.

Variante da provare: Aggiungi un tasso di interesse semplice annuo e ricalcola.